VSCLMS

College Leave Management System

Professional Developer Guide

Version 1.0 | 26 October 2026

1. Project Title

VSCLMS — College Leave Management System: A Secure, Auditable, Day-Wise Leave and Workload Management Web Application.

2. Abstract

VSCLMS is a role-based web application for managing college staff leave from application through workload handover, HOD review, Principal decision, final balance posting, and reporting. It supports STAFF, HOD, and PRINCIPAL roles; past and future leave requests; full-day and half-day leave; Sunday and holiday exclusion; automatic paid-leave/LWP classification; per-day workload assignment and transfer; cancellation before approval; and day-level reversal after approval. The system is designed around a day-wise LeaveDay ledger so operational decisions, leave consumption, LWP, reversals, and reports can be traced to individual dates. All meaningful actions must be authorized, transactional where balance/workflow consistency is affected, and recorded in an audit trail. The developer must build and verify the application from an empty project, not assume that any module or code already exists.

3. Project Objectives

- Digitize and standardize college staff leave processing with explicit role-based approval.
- Track every requested date independently through LeaveDay records.
- Prevent duplicate/overlapping applications and workload conflicts.
- Calculate paid leave and Leave Without Pay (LWP) without blocking an application solely for insufficient balance.
- Maintain accurate balances with final-approval-only deductions and controlled reversal.
- Provide reliable notifications, audit history, and role-scoped operational reports.
- Deliver a maintainable, testable Django application with a clear service layer and documented deployment.

4. Scope and Core Principles

- Roles: STAFF, HOD, PRINCIPAL. Only one Principal may exist.
- All leave dates, past and future, follow the defined workflow, except the explicit rule that a Principal's own application is auto-approved.
- LeaveDay is the single source of truth for day counts, paid/LWP totals, deduction, reversal, and reports.
- No balance deduction before final approval; LWP is never deducted from paid balance.
- No sandwich rule: Sundays and holidays are not counted. Do not assume Saturday is a holiday unless the college calendar says so.
- Submitted applications are editable only while WORKLOAD_PENDING; deletion is allowed only if no action has been taken.
- Rejected and soft-deleted applications do not block overlap validation.
- Every sensitive state transition must be checked server-side and logged.

5. Recommended Technology Stack

Layer	Recommendation	Reason / rule
Backend	Python 3.12+ and a supported Django LTS release	Choose and pin exact supported versions at kickoff.
Database	SQLite for local prototype; PostgreSQL for production(skip for now).	Production approval/balance concurrency needs reliable row locking and constraints.
Frontend	Django templates, HTML5, CSS, Bootstrap 5, vanilla JavaScript or jQuery	Fast delivery; no separate SPA required.
Authentication	Django authentication and sessions	Use password hashing, CSRF protection, and server-side permissions.
Business logic	Dedicated services.py modules and transaction.atomic()	Keep workflow and balance logic out of views/templates.
Reporting	Django ORM aggregation; CSV export initially	Report calculations must derive from LeaveDay.
Notifications	Database-backed notification records plus approved email adapter	Persist event status; send after transaction commit.
Testing	Django TestCase or pytest-django; production-like DB for locking tests	Test rules, permissions, workflows, and concurrency.

Pin exact framework/package versions in requirements.txt. Do not upgrade dependencies casually during implementation.

6. System Modules

Module	Main features
A. Authentication & Staff	Login/logout, active account status, profile, role, department, designation, role-protected navigation.
B. Academic Configuration	Academic year creation/activation, department/HOD mapping, designation and leave type setup.
C. Holiday Calendar	Holiday management by academic year; soft deletion; adjustment handling for existing LeaveDay records.
D. Leave Balance	Balance setup per staff + leave type + academic year; used/remaining display; locked final deduction and paid reversal restoration.
E. Leave Application	Past/future leave, leave type, reason, day fraction(s), workload requirement; timeline and day breakdown.
F. Day-Wise Engine	One LeaveDay per date; classify working day/Sunday/holiday/counting; calculate fraction and paid/LWP allocation.
G. Workload Management	Per-day assignment, partial assignments, accept/reject, conflict checks, workload inbox, transfer.
H. HOD Workflow	Department-scoped review queue, approve/reject with remarks, audit and notifications.
I. Principal Workflow	Institution-level review, final approval/rejection, balance posting, permitted reversals.
J. Cancellation & Reversal	Pre-approval cancellation under the no-action rule; post-approval day-level reversal for eligible future/unutilized dates.
K. Notifications	Events for application, assignment, workload action, approval/rejection, and reversal.
L. Audit Trail	Actor, action, timestamp, remarks, affected leave/day/workload, optional before/after values.
M. Reports & Dashboard	Staff summary, department usage, monthly usage, LWP, pending queues, workload distribution.

7. Roles and Permission Matrix

Action	STAFF	HOD	PRINCIPAL	Authorized administrator
Apply for leave	Yes	Yes	Yes	Only if also assigned an eligible staff profile
Act on assigned workload	Yes	Yes	Yes if assigned	Only if also an eligible assignee
HOD-stage decision	No	Assigned department only; never own leave	Not a substitute for HOD unless configured	No by default
Principal-stage decision	No	No	Yes; own-application exception applies	No by default
Configure academic year/holiday/balance	No	No by default	Only if institution policy grants it	Yes
Institution-wide reports	No	Department-scoped	Yes	As authorized

Enforce permissions for every view and state-changing service. UI hiding alone is not security.

8. Main Workflows

8.1 Standard leave lifecycle

Step	Action	Expected result
1	Applicant submits leave	Validate active year, date range, leave type, overlap, permissions; create LeaveApplication and one LeaveDay per date.
2	Classify each date	Set working-day, Sunday, holiday, counted, fraction, workload-required, and provisional paid/LWP flags.
3	Workload stage	Create/collect required assignments per date; prevent self-assignment and duplicates.
4	Workload decisions	Any required workload rejection rejects leave; advance only when the defined completion rule is satisfied.
5	HOD review	Authorized HOD approves to PRINCIPAL_PENDING or rejects to REJECTED.
6	Principal review	Principal approves/rejects; final approval rechecks balance inside a transaction.
7	Final approval	Deduct paid counted fractions once, never deduct LWP, mark relevant LeaveDays APPROVED, update summary snapshots, audit and notify.

8	After approval	Allow only permitted future/unutilized LeaveDay reversal; restore paid fraction only and preserve audit history.

8.2 Status transitions

Current status	Permitted transition	Condition
WORKLOAD_PENDING	HOD_PENDING	Workload requirement satisfied or workload explicitly not required under approved rule.
WORKLOAD_PENDING	REJECTED	Workload rejection triggers rejection under the defined rule.
HOD_PENDING	PRINCIPAL_PENDING	Authorized HOD approves; actor is not applicant.
HOD_PENDING	REJECTED	Authorized HOD rejects.
PRINCIPAL_PENDING	APPROVED	Principal approves; transaction revalidates and deducts paid balance.
PRINCIPAL_PENDING	REJECTED	Principal rejects.
Pre-approval state	CANCELLED	Only when cancellation eligibility/no-action condition is met.
APPROVED	LeaveDay REVERSED	Only eligible future/unutilized days; day-level transaction and audit.

The Principal's own application is the explicit exception to the normal workflow and is auto-approved. Confirm whether this exception bypasses workload and HOD stages; do not invent that behavior.

8.3 Paid leave and LWP flow

1. Read the relevant balance for applicant, leave type, and academic year.
2. Consider only LeaveDay records where is_counted=True.
3. Compare each counted day's fraction (FULL=1.0; HALF_AM/HALF_PM=0.5) with available balance.
4. Set is_lwp=False if the entire fraction is covered; otherwise set is_lwp=True. Insufficient balance must not block application.
5. At final approval, lock the balance row and recalculate allocation to account for concurrent approvals.
6. Deduct only approved, counted, non-LWP fractions. Never deduct LWP and never deduct before final approval.

Recommended decision for approval: allocate paid leave chronologically and mark a day fully LWP if its full fraction cannot be covered; obtain product-owner confirmation before treating this as final policy.

8.4 Cancellation versus reversal

Operation	When	Balance impact	Record handling
Cancel	Before approval and only if no-action rule is met	None	Application status CANCELLED; audit; no deduction/restoration.
Reverse a day	After approval, for eligible future/unutilized LeaveDay only	Restore fraction only if paid; never restore LWP	Mark LeaveDay REVERSED, retire associated workload without losing history, update summaries and audit.

8.5 Workload transfer

- Allowed only when old workload is ACCEPTED, leave is APPROVED, and LeaveDay.date >= today.
- Old assignment becomes TRANSFERRED; create a new PENDING assignment and link transferred_from.
- Validate new assignee active status, no self-assignment, duplicates, and date conflicts.
- Perform the change atomically; notify affected users and record WORKLOAD_TRANSFERRED.

9. Data Model Requirements

Entity	Core fields / constraints
AcademicYear	name unique, start_date, end_date, is_active, created_at; start_date < end_date; one active year.
Department	name unique, optional HOD profile; HOD role and department must match.
Designation	unique title.
StaffProfile	one-to-one User, department, optional designation, role STAFF/HOD/PRINCIPAL, is_active; one Principal.
Holiday	date, name, academic_year, is_deleted/deleted_at/deleted_by, created_at; unique date + academic year.
LeaveType	unique name, description, is_active.
StaffLeaveBalance	staff, leave_type, academic_year, total, used; unique combination; used nonnegative.
LeaveApplication	applicant, academic_year, leave_type, dates, reason, workload flag, status, summary snapshots, soft-delete fields.

LeaveDay	leave FK, date, working/holiday/Sunday/counted flags, status PENDING/APPROVED/REVERSED, fraction, workload_required, is_lwp; unique leave + date.
LeaveWorkload	leave_day, assigned_staff, status PENDING/ACCEPTED/REJECTED/TRANSFERRED, remarks, transferred_from, is_deleted.
LeaveApprovalLog	leave, action_by, action, remarks, created_at, is_deleted; indexed by leave and timestamp.

LeaveDay is authoritative. LeaveApplication totals (balance_before, balance_after, total_days_counted, total_days_paid, total_days_lwp) are summary snapshots only. Derive official counts and reports from LeaveDay.

10. Critical Design Rules and Implementation Gaps

- Use database transactions and row locking for final approval, balance deduction, reversal, and workload transfer. PostgreSQL is recommended for production concurrency.
- Use database constraints for uniqueness and invariants; handle IntegrityError for concurrent writes.
- The supplied ActiveManager filters is_deleted=False, but not every baseline model has is_deleted. Attach it only to compatible models or add fields through an approved migration.
- The requested LeaveDay (staff, date) index cannot be created directly from the baseline because LeaveDay has no staff field; optimize with supported indexes/joins or approve a denormalized staff field.
- The requested workload (assigned_staff, date) index needs a design decision because LeaveWorkload stores leave_day, not date.
- Django save() does not automatically run full_clean(); bulk_create/update bypass model save and validation. Validate in services and tests.
- FloatField is risky for half-day arithmetic. Prefer DecimalField or integer half-day units after confirming migration/reporting implications.
- Transferred workload history may conflict with unique_together (leave_day, assigned_staff). Define active-assignment uniqueness separately from transfer history.
- The baseline does not implement the complete workflow, notifications, day generation, LWP allocation, approval deduction, reversal, or concurrency handling; these must be built.

11. Do’s and Don’ts

DO

- Start from an empty repository and make the development environment reproducible.
- Build models, migrations, services, views, templates, and tests as fresh deliverables.
- Put state-changing business logic in services and wrap related writes in transactions.
- Validate status, role, department scope, dates, balance, workload and overlap inside the transaction.
- Derive all calculations from LeaveDay and show counted/paid/LWP/reversed figures separately.
- Use audit logs for approvals, rejections, cancellation, workload actions, holiday adjustments, balance changes, and reversals.
- Use seeded development data only for local testing; never present demo records as real institutional records.
- Keep secrets in environment variables, enforce HTTPS in production, and back up the database.
- Write automated tests before considering a module complete.

DON’T

- Do not assume any previous code, database, screens, or deployment exists.
- Do not calculate leave consumption from the date span on LeaveApplication.
- Do not deduct balance when the request is submitted or deduct LWP.
- Do not auto-approve past leave or skip workflow stages, except the explicit Principal-own-leave exception.
- Do not allow applicants to approve their own leave outside that exception.
- Do not trust client-side validation or hide-only permission controls.
- Do not let a holiday edit silently change paid balance after approval.
- Do not hard-delete audit history or erase transferred workload lineage.
- Do not send notifications before transaction commit or let delivery failure corrupt a committed decision.
- Do not use SQLite locking behavior as proof that production concurrency is safe.

12. Required UI Pages

Page	Required content
Login	Secure sign-in, validation, logout.
Dashboard	Role-based pending queue, balances, upcoming leave, assigned workloads, recent alerts.
Apply Leave	Active academic year, leave type, date range, reason, day fractions, workload option, day-by-day preview.
My Leaves / Detail	Status, timeline, LeaveDay table, paid/LWP split, workloads, audit history.
Workload Inbox	Assignments by date; accept/reject with remarks.
HOD Queue	Department-scoped requests; day details; approve/reject with remarks.
Principal Queue	HOD-approved requests; approve/reject; final balance effect preview.
Calendar / Holidays	Academic-year calendar and holiday management.
Staff / Balances	Staff role and department; balance setup and usage.
Reports	Staff summary, department usage, monthly report, LWP, workload distribution.
Notifications	Unread/read alerts linking to relevant record.

13. Notifications and Audit Events

Event	Minimum requirement
LEAVE_CREATED	Log successful submission; notify relevant parties.
LEAVE_DAY_CREATED / UPDATED	Track generated/changed day records.
WORKLOAD_ASSIGNED	Log and notify assignee.
Workload accepted/rejected	Log decision and remarks; notify applicant/workflow.
HOD/Principal decision	Record actor, decision, time, remarks; notify applicant.
LEAVE_APPROVED / LEAVE_REJECTED	Record outcome; approval includes paid/LWP summary.
LEAVE_CANCELLED	Record cancellation.
HOLIDAY_ADJUSTED	Record affected dates and changes without automatic balance adjustment after approval.
WORKLOAD_TRANSFERRED	Preserve old/new assignment lineage.
LEAVE_PARTIALLY_REVERSED	Record day-level reversal and any paid balance restoration.

Add explicit audit action values where needed and document them; do not silently omit these events.

14. Reports

- Staff leave summary by academic year and leave type.
- Department-wise usage and pending applications.
- Monthly leave report grouped by LeaveDay.date.
- LWP tracking by staff, department, leave type, date, and fraction.
- Workload distribution by assignee, date, status, and transfer history.

Reports must distinguish requested, pending, approved, paid, LWP, and reversed days. Exclude reversed paid fractions from consumed paid leave. Scope results by role and derive day totals from LeaveDay.

15. Quick Development Plan

Phase	Deliverable	Completion gate
1 — Project foundation	Django project, apps, settings, auth, base template, logging, migrations.	Clean install and login work from documented steps.
2 — Setup/admin	Academic years, departments, staff profiles, leave types, holidays, balances.	Constraints and role/department validation tested.
3 — Day engine	LeaveDay creation, date classification, fractions, paid/LWP preview.	Unit tests cover Sundays, holidays, half-days, balance cases.
4 — Workload	Assignment, decision, conflict check, transfer.	No self-assignment, duplicates or invalid transfers.
5 — Approval workflow	HOD and Principal queues, decisions, stage enforcement.	All valid/invalid transitions tested.
6 — Balance and reversal	Atomic final deduction, cancellation, day reversal, holiday changes.	No double deduction/restoration under retries/concurrency tests.
7 — Notifications and reports	Event records/delivery, audit trail, dashboards, CSV reports.	All required triggers audited; totals reconcile to LeaveDay.

16. Acceptance Criteria

- Application can be installed and run from scratch using README and pinned requirements.
- All listed roles and authorization boundaries work server-side.
- Every leave date has exactly one LeaveDay record.
- Past and future leave follow the same workflow, subject to the Principal exception.
- Insufficient balance produces LWP rather than blocking the application.
- Paid balance is deducted exactly once at final approval; LWP is never deducted.
- Holiday adjustment, cancellation, reversal, and transfer preserve history and follow distinct rules.
- Overlap and workload conflicts are checked safely under concurrent requests.
- All required notifications and audit events are generated.
- All required reports derive from LeaveDay and reconcile with test fixtures.
- Automated tests cover business rules, permissions, workflow transitions, and concurrency-sensitive operations.

17. Decisions to Confirm Before Finalizing Affected Features

7. Paid/LWP allocation order across multiple dates when balance is insufficient.
8. Whether multiple half-days can be included in one application and how mixed full/half-day requests are entered.
9. Whether any weekday besides Sunday can be non-working; confirm Saturday/college calendar rules.
10. Whether a leave request may cross academic-year boundaries.
11. Exact definition of workload completion when assignments are partial or absent.
12. Whether Principal auto-approval bypasses workload and HOD stages.
13. What qualifies as 'no action taken' for cancellation/deletion.
14. Definition of 'future/unutilized' for reversal, especially on the current date.
15. Notification channels, recipients, and message templates.
16. Production database and deployment target.
17. Additional audit event names and whether before/after snapshots are required.
18. Whether administrative balance adjustments are allowed and how they are approved/audited.

18. Final Instruction to the Developer

Do not claim that any prior implementation exists or reuse unverified code. First create the repository, environment, database, migrations, and test baseline. Implement one module at a time in the phase order above. For each phase, deliver source code, migrations, tests, setup instructions, and a short completion note. Where a business rule is not explicit, document the question and obtain approval rather than guessing. Production release is complete only when acceptance criteria and automated tests pass.